

IsPilot + Copilot Studio + Teams Integration Design

1. Objetivo

Diseñar e implementar la capa de integración empresarial necesaria para exponer IsPilot fuera de Vertex AI Agent Engine y hacerlo disponible a través de Microsoft Teams y Microsoft Copilot Studio.

Este documento asume que IsPilot ya está desplegado y es un sistema productivo. Por tanto, el objetivo principal no es modificar IsPilot ni la lógica multiagente, sino construir la capa de integración, seguridad, observabilidad y acceso para que el agente pueda ser consumido desde canales empresariales.

2. Principio arquitectónico

Regla central

IsPilot debe seguir siendo el motor de negocio.

No se debe mover la lógica de negocio ni los agentes especialistas a Copilot Studio.

Copilot Studio y Teams deben actuar como:

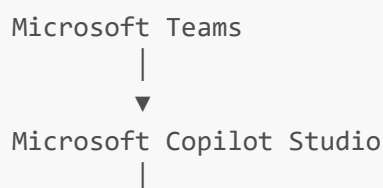
- capa de usuario,
- capa de autenticación,
- capa de experiencia conversational,
- capa de orquestación de canales,
- capa de presentación del resultado final.

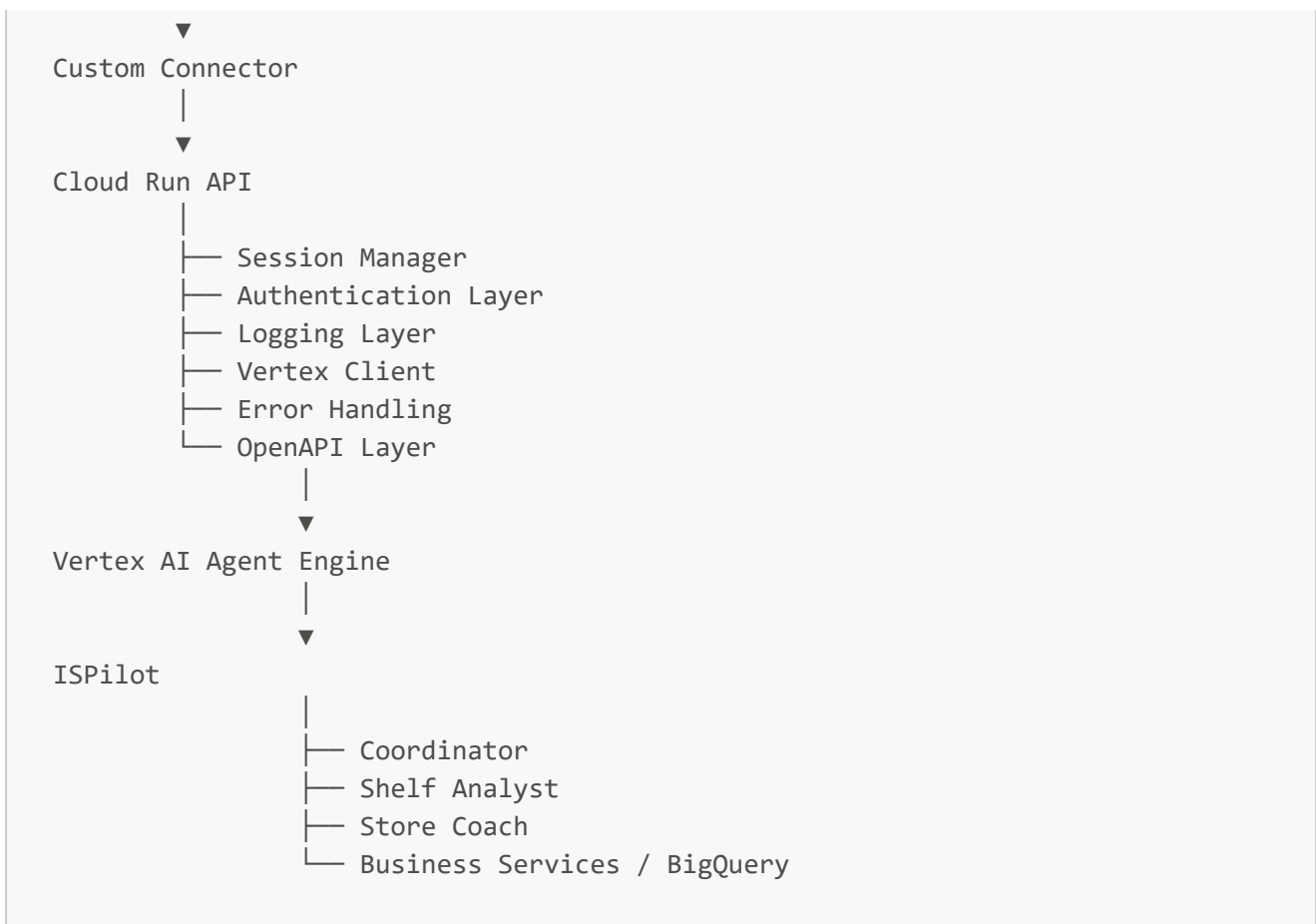
Patrón recomendado

IsPilot es el backend inteligente y la fuente de verdad para:

- análisis de métricas de tienda,
 - diagnóstico operativo,
 - recomendaciones de acción,
 - delegación a especialistas,
 - acceso a BigQuery y servicios de negocio,
 - gobernanza del conocimiento del sistema.
-

3. Arquitectura objetivo





4. Estado actual validado

Los siguientes componentes ya existen y han sido validados:

- ☒ Vertex AI Agent Engine
- ☒ Coordinator Agent
- ☒ Shelf Analyst
- ☒ Store Coach
- ☒ BigQuery integration
- ☒ Secret Manager integration
- ☒ Multi-agent routing
- ☒ Vertex Playground
- ☒ REST API access
- ☒ Session creation
- ☒ Query execution
- ☒ Tool calling
- ☒ BigQuery retrieval

Esto significa que la capa de inteligencia ya está funcionando. Lo que falta es la capa de integración empresarial para exponerla a sistemas externos sin tocar la lógica del agente.

5. Configuración productiva

Proyecto

- Project: corp-stro-salesinventory-prod
- Region: us-central1

Resource actual del agente

- Resource: projects/390358249123/locations/us-central1/reasoningEngines/5375474415045705728
- Reasoning Engine ID: 5375474415045705728
- Query endpoint: <https://us-central1-aiplatform.googleapis.com/v1/projects/390358249123/locations/us-central1/reasoningEngines/5375474415045705728:query>

Contrato API validado

Crear sesión

```
POST
{
  "classMethod": "create_session",
  "input": {
    "user_id": "test-user"
  }
}
```

Consultar al agente

```
POST
{
  "classMethod": "stream_query",
  "input": {
    "user_id": "test-user",
    "session_id": "SESSION_ID",
    "message": "How is Talca Colin performing?"
  }
}
```

Este es el contrato de integración técnico que debe encapsularse detrás de la capa Cloud Run/API pública.

6. Diseño de la capa de integración

6.1 Capa 1: Vertex Agent Engine

Esta capa es la runtime de IsPilot. No se modifica.

Debe quedar encapsulada detrás de una interfaz estable.

6.2 Capa 2: Cloud Run API

Cloud Run será la capa pública de integración y la parte crítica para normalizar acceso, seguridad y control operativo.

Deberá exponer endpoints como:

- GET /health
- POST /chat
- POST /session
- POST /session/refresh

6.3 Capa 3: Custom Connector en Copilot Studio

Copilot Studio invoca la API externa vía custom connector. No debe consumir directamente Vertex ni internals del agente.

6.4 Capa 4: Teams

Microsoft Teams será el canal final, pero no el sistema de lógica ni de negocio. Será la experiencia de usuario en el flujo de trabajo.

7. Modelo operacional recomendado

Flujo de ejecución

```
Teams user
↓
Copilot Studio
↓
Custom Connector
↓
Cloud Run API
↓
Vertex AI Agent Engine
↓
ISPilot
↓
Coordinator -> Shelf Analyst / Store Coach
↓
BigQuery and business services
```

Ventajas

- Aisla el backend de los canales.
- Permite seguridad y control centralizado.

- Facilita observabilidad y trazabilidad.
- Permite evolucionar Teams o Copilot sin romper IsPilot.
- Mantiene la arquitectura productiva clara y gobernable.

8. Estructura recomendada del repositorio API

```
ispilot-api/  
├── app/  
│   ├── main.py  
│   ├── api/  
│   │   └── chat.py  
│   ├── services/  
│   │   ├── vertex_client.py  
│   │   ├── session_service.py  
│   │   └── auth_service.py  
│   ├── models/  
│   │   ├── chat_request.py  
│   │   ├── chat_response.py  
│   │   └── errors.py  
│   ├── config/  
│   │   ├── settings.py  
│   │   └── secrets.py  
│   ├── utils/  
│   │   ├── logging.py  
│   │   └── healthcheck.py  
│   └── db/  
│       └── firestore_client.py  
├── requirements.txt  
├── Dockerfile  
├── openapi.yaml  
├── README.md  
├── .env.example  
└── deploy.sh
```

9. Diseño funcional de la API

9.1 Endpoint de salud

GET /health

Respuesta:

```
{  
  "status": "healthy"
```

```
}
```

9.2 Endpoint de chat

POST /chat

Request:

```
{
  "user_id": "sebastian",
  "message": "How is Talca Colin performing?"
}
```

Response:

```
{
  "answer": "Talca Colín está mostrando ...",
  "response_source": "ispilot",
  "session_id": "abc123",
  "status": "ok"
}
```

9.3 Endpoint de sesión

POST /session

Request:

```
{
  "user_id": "sebastian"
}
```

Response:

```
{
  "session_id": "abc123",
  "status": "created"
}
```

10. Diseño del cliente Vertex

Debemos encapsular todo lo específico de Vertex en una clase dedicada.

VertexAgentClient

Responsabilidades:

- `create_session()`
- `stream_query()`
- parsear respuestas de Vertex
- mapear errores de Vertex a errores de negocio
- ocultar `classMethod`, `stream_query`, `create_session` y el endpoint de reasoning engine al resto del sistema

Ejemplo de abstracción

```
class VertexAgentClient:
    def create_session(self, user_id: str) -> str:
        ...

    def stream_query(self, user_id: str, session_id: str, message: str) -> dict:
        ...
```

Regla importante

El resto de la plataforma no debe saber:

- que existe `reasoningEngines`
- que existe el endpoint `:query`
- que usa `classMethod`
- que hay un `stream_query`
- que hay una API específica de Vertex

Todo eso queda encapsulado en una sola capa.

11. Diseño de sesiones con Firestore

Se recomienda usar Firestore para almacenar sesiones por usuario.

Colección

`user_sessions`

Esquema sugerido

```
{
  "user_id": "sebastian",
  "session_id": "abc123",
  "created_at": "2026-08-25T10:00:00Z",
  "last_used_at": "2026-08-25T10:05:00Z",
  "status": "active",
  "environment": "dev"
}
```

Responsabilidades

- crear sesión cuando no exista,
- reutilizar sesión existente,
- invalidar o refrescar sesiones expiradas,
- relacionar usuario con sesión en Vertex,
- mantener trazabilidad por usuario.

12. Seguridad

12.1 Modelo recomendado

- Microsoft Entra ID para identidad del usuario en Teams/Copilot Studio.
- Cloud Run con autenticación protegida.
- IAM para permisos del servicio Cloud Run hacia Vertex y Firestore.
- Secret Manager para datos sensibles.

12.2 Recomendaciones

- no anonymous access,
- no secretos hardcodeados,
- acceso por identidad y no por tokens expuestos al usuario final,
- controlar por entorno (DEV / QAS / PRD),
- aplicar trazabilidad de request por `user_id` y `session_id`.

13. Logging, observabilidad y monitoreo

13.1 Logging requerido

Registrar para cada request:

- `user_id`
- `session_id`
- `timestamp`
- `duration_ms`
- `agent_name`
- `success / failure`

- `error_code`
- `request_summary`

13.2 Observability recomendada

- Cloud Logging
- Cloud Monitoring
- métricas de latencia
- tasa de errores
- número de sesiones activas
- fallas de Vertex
- timeout y invalid requests

14. Manejo de errores

Contrato estándar

```
{
  "error_code": "VERTEX_TIMEOUT",
  "error_message": "The request to the agent timed out. Please retry in a few moments."
}
```

Casos a manejar

- Vertex failures
- Firestore failures
- timeout failures
- invalid requests
- auth failures
- missing session
- malformed responses

15. Secret Manager

Configurar en Secret Manager:

- engine id
- project id
- region
- token / service account configuration if needed
- future enterprise secrets

Regla:

- no guardar secretos en código,
 - no guardar secretos en `env` de repositorio,
 - no usar archivos `.json` en repositorios para producción.
-

16. API documentation

FastAPI genera automáticamente Swagger y OpenAPI.

La documentación esperada debe incluir:

- POST /chat
 - GET /health
 - definiciones de schema
 - ejemplos de request y response
 - autenticación y seguridad
-

17. Sprint 1 — Foundation + Cloud Run API

Objetivo

Crear una API de fachada sobre Vertex Agent Engine.

Entregables

Deliverable 1: nuevo servicio / repositorio

Servicio recomendado:

- `ispilot-api`

Estructura sugerida:

```
app/  
├─ main.py  
├─ api/  
│   └─ chat.py  
├─ services/  
│   ├── vertex_client.py  
│   └─ session_service.py  
├─ models/  
├─ config/  
└─ utils/
```

Deliverable 2: VertexAgentClient

Responsabilidades:

- create_session()
- stream_query()

Debe encapsular todo lo específico de Vertex.

Deliverable 3: Session Service con Firestore

Colección:

- user_sessions

Responsabilidades:

- crear sesión si no existe,
- reutilizar sesión existente,
- manejar expiración.

Deliverable 4: Health endpoint

```
GET /health
```

Deliverable 5: Chat endpoint

```
POST /chat
```

Request:

```
{
  "user_id": "sebastian",
  "message": "How is Talca Colin performing?"
}
```

Response:

```
{
  "answer": "...",
  "response_source": "ispilot"
}
```

Deliverable 6: despliegue en Cloud Run

Documentar:

- proceso de build,
 - comandos `gcloud`,
 - deployment, configuración del servicio,
 - variables de entorno,
 - scripts de despliegue.
-

18. Sprint 2 — Security, Logging & Observability

Objetivo

Preparar la API para uso empresarial y productivo.

Entregables

Deliverable 1: estrategia de autenticación

Recomendación:

- Cloud Run con autenticación controlada,
- IAM para servicio,
- autorización basada en identidad,
- protección de acceso a la API.

Deliverable 2: logging

Usar Cloud Logging para registrar:

- `user_id`
- `session_id`
- request payload summary
- `timestamp`
- `duration`
- `agent invoked`
- `success/failure`

Deliverable 3: manejo de errores

Contrato estándar:

```
{
  "error_code": "",
  "error_message": ""
}
```

Deliverable 4: Secret Manager

Mover configuraciones sensibles a Secret Manager.

Deliverable 5: Swagger / OpenAPI

Generar automáticamente con FastAPI.

19. Sprint 3 — Copilot Studio Integration

Objetivo

Conectar Copilot Studio con la API de IsPilot.

Entregables

Deliverable 1: OpenAPI Specification

Generar `openapi.yaml` con:

- GET /health
- POST /chat
- examples de request/response
- autenticación definitions

Deliverable 2: Custom Connector

Crear `ISPilot Connector` con responsabilidades:

- invocar Cloud Run API,
- enviar el mensaje del usuario,
- recibir y mostrar la respuesta.

Deliverable 3: guía de configuración en Copilot Studio

Pasos necesarios:

1. Crear connector.
2. Importar OpenAPI.
3. Configurar autenticación.
4. Configurar connector.
5. Probar connector.
6. Publicar Copilot.

Deliverable 4: validación funcional

Validar:

- Pregunta: "How is Talca Colin performing?"
- Expected route: Copilot -> Cloud Run -> Vertex -> Coordinator -> Shelf Analyst

- Pregunta: "How can Talca Colin improve?"
 - Expected route: Copilot -> Cloud Run -> Vertex -> Coordinator -> Store Coach
-

20. Sprint 4 — Teams Integration

Objetivo

Hacer IsPilot disponible dentro de Microsoft Teams.

Entregables

Deliverable 1: publicación en Teams

Documentar:

- publicación,
- configuración del Team,
- permisos,
- flujo de acceso.

Deliverable 2: propagación de contexto del usuario

Pasar:

- user identity,
- Teams user,
- session context,
- request metadata,
- user/tenant mapping,
- session reuse strategies.

Deliverable 3: estrategia de autenticación con Microsoft Entra ID

Diseño sugerido:

```
graph TD
    User --> MicrosoftEntraID[Microsoft Entra ID]
    MicrosoftEntraID --> Teams
    Teams --> CopilotStudio[Copilot Studio]
    CopilotStudio --> CloudRunAPI[Cloud Run API]
    CloudRunAPI --> VertexAgentEngine[Vertex Agent Engine]
    VertexAgentEngine --> ISPilot
```

Deliverable 4: audit layer

Persistir:

- user
- question
- timestamp
- session_id
- agent used
- response status

Deliverable 5: monitoreo

Implementar:

- Cloud Monitoring
- Cloud Logging
- alertas por fallas y latencia

21. Criterios de éxito productivos

Criterio 1

Un usuario en Microsoft Teams puede preguntar:

"How is Talca Colin performing?"

y recibir una respuesta basada en Shelf Analyst.

Criterio 2

Un usuario en Microsoft Teams puede preguntar:

"How can Talca Colin improve?"

y recibir una respuesta basada en Store Coach.

Criterio 3

El flujo debe ser:

```
Teams
↓
Copilot Studio
↓
Custom Connector
↓
```

```
Cloud Run API
↓
Vertex Agent Engine
↓
ISPilot
```

Sin utilizar Vertex Playground.

22. Entregables esperados

Se deben entregar finalmente:

1. Architecture diagram
 2. Cloud Run design
 3. Vertex client design
 4. Firestore session design
 5. OpenAPI specification
 6. Python implementation
 7. Cloud Run deployment guide
 8. Copilot Studio connector guide
 9. Teams publishing guide
 10. Security design
 11. Monitoring strategy
 12. Audit strategy
 13. Production readiness checklist
-

23. Recomendación final

La solución correcta para IsPilot es la siguiente:

- IsPilot = motor de negocio y agente especialista
- Vertex Agent Engine = runtime de ejecución del agente
- Cloud Run API = capa de integración segura y gobernada
- Copilot Studio = capa de experiencia y orquestación
- Teams = canal de usuario final
- Firestore = sesión y contexto del usuario
- Secret Manager = secretos y configuración sensible
- Cloud Logging / Monitoring = operabilidad y observabilidad

Esto produce una arquitectura empresarial real, escalable y mantenible, sin romper la lógica actual de IsPilot ni la infraestructura ya validada.

24. Siguientes pasos inmediatos

1. Implementar la API Cloud Run con FastAPI.

2. Encapsular Vertex en `VertexAgentClient`.
3. Crear el servicio de sesiones con Firestore.
4. Crear endpoints `/health` y `/chat`.
5. Desplegar en DEV.
6. Validar con real requests a Vertex.
7. Conectar custom connector en Copilot Studio.
8. Publicar en Teams luego de validar QAS.
9. Cerrar PRD con seguridad, logs y observabilidad.

Este es el camino correcto para pasar de una prueba técnica a una integración empresarial productiva.